
Efficient LLM Serving using vLLM and PagedAttention

A technical review of memory-optimized inference for large language models through virtual memory management techniques.

PRESENTER

Nicholas Lisle

COURSE

CAP6614

INSTITUTION

University of Central Florida

TERM

Spring 2026

01 / 14

MOTIVATION: SCALING AI



The Cost of VRAM

LLM inference is memory-bound. GPU VRAM is expensive and limited, making efficiency the top priority for production systems.



Serving Scalability

Poor throughput limits concurrent users. To scale to millions, we need engines that can pack more requests per GPU.



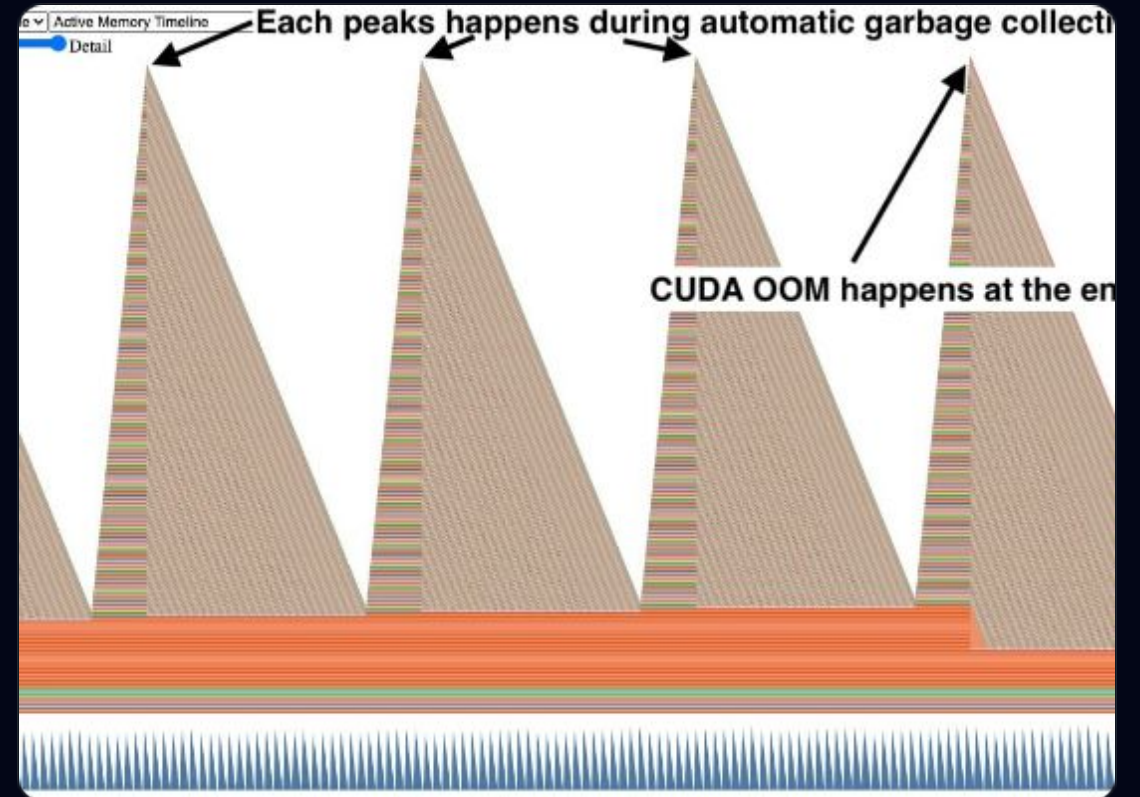
User Experience

High latency ruins interactive applications. Optimizing memory avoids the recomputation overhead that slows down tokens.

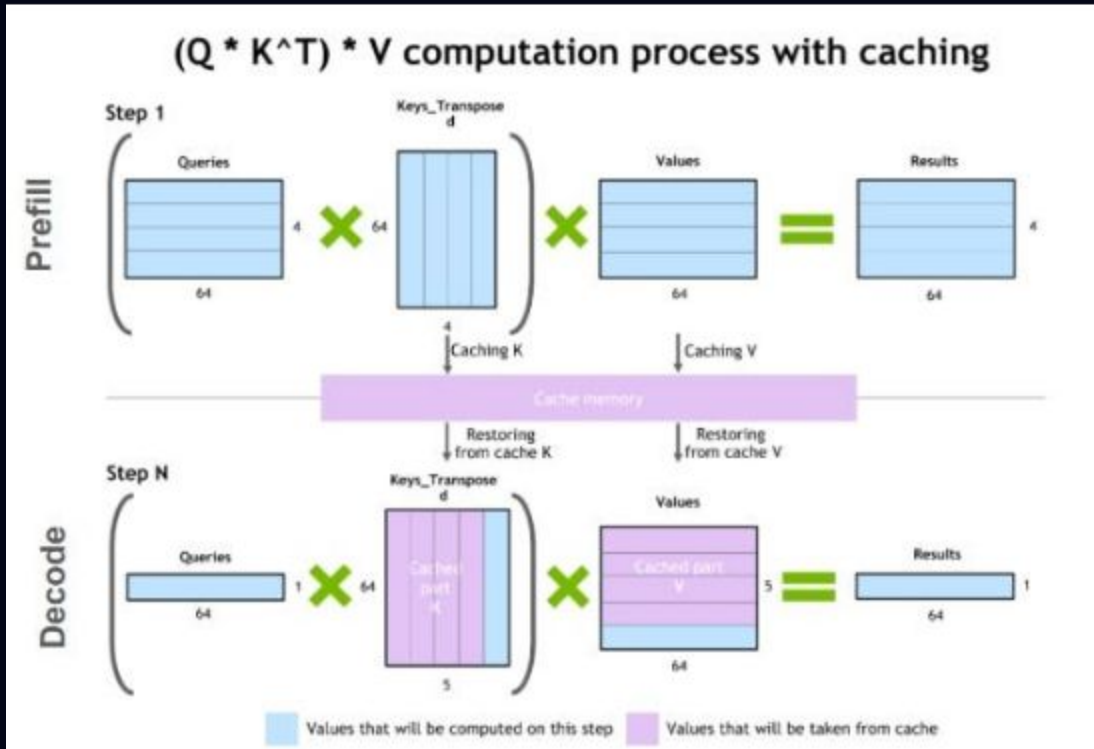
TRADITIONAL INFERENCE BOTTLENECK

Fragmentation & Waste

- ⚠ **Contiguous Allocation:** Standard pipelines reserve long, rigid blocks of VRAM upfront.
- ✂ **Internal Fragmentation:** Reserved memory that is never used by the generated sequence.
- 🧩 **External Fragmentation:** Gaps between allocations that are too small to fit new requests.
- 🚫 **OOM Risks:** Static over-allocation leads to premature out-of-memory errors.



WHAT IS THE KV CACHE?



The Persistence of State

-  **Storage:** Stores attention keys and values from previous tokens to avoid recomputation.
-  **Speed:** Essential for fast autoregressive generation (token-by-token).
-  **VRAM Usage:** Grows linearly with sequence length, becoming the primary memory consumer.
-  **Inefficiency:** Traditional caching doesn't allow memory to be shared or resized dynamically.

SOLUTION: PAGED ATTENTION



OS-Style Paging

Divide the KV cache into fixed-size physical blocks. Logical sequences are mapped to these non-contiguous pages.



Flexible Mapping

A Block Table tracks logical tokens to physical locations, allowing on-demand allocation without fragmentation.



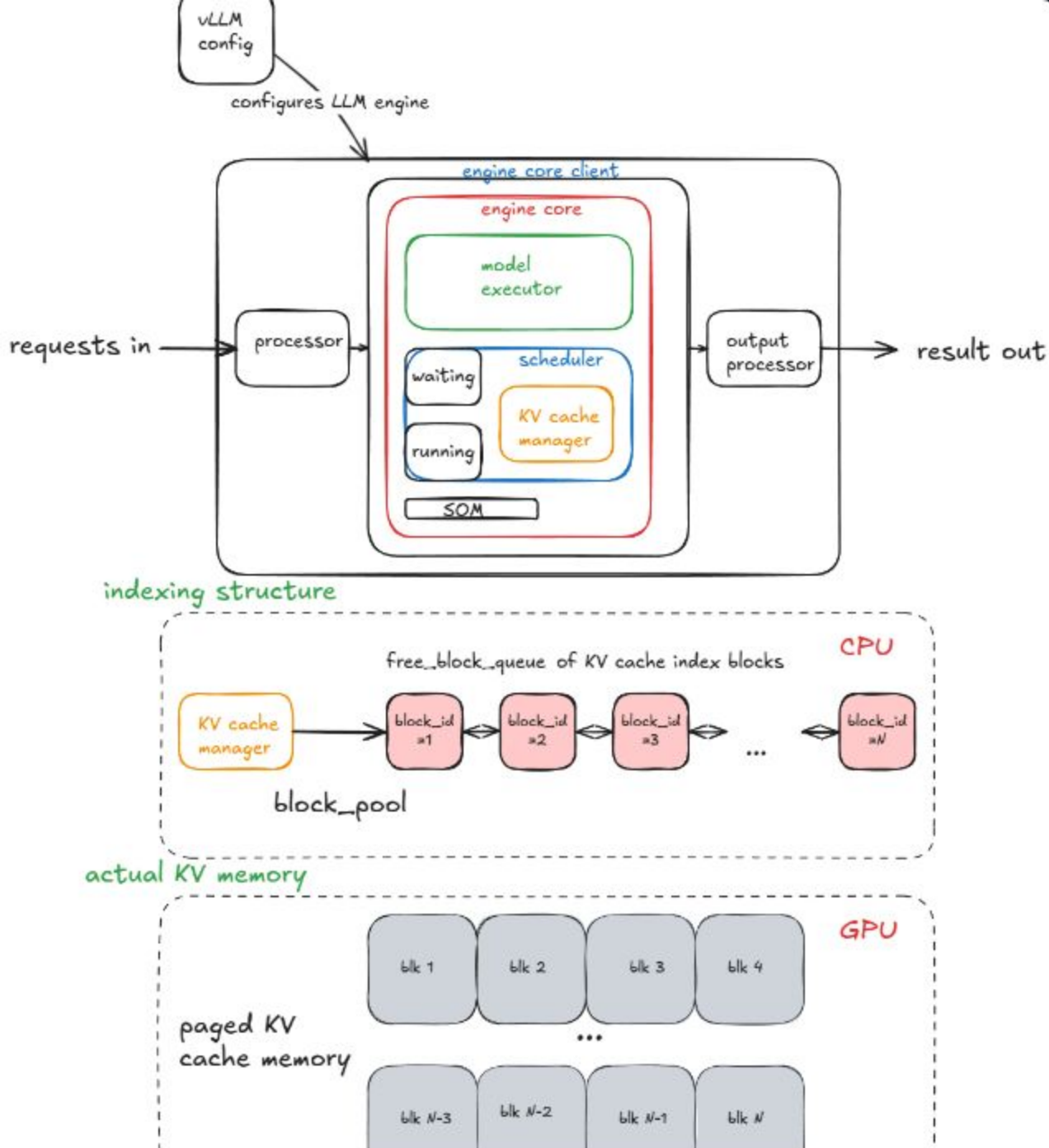
Efficient Sharing

Parallel requests (like beams) can share identical pages, dramatically reducing the footprint of diverse outputs.

VLLM ARCHITECTURE

System Orchestration

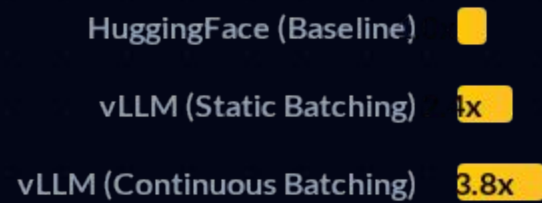
- 🔧 **Execution Engine:** Optimized kernels for PagedAttention operations.
- 📋 **Continuous Batching:** Schedules requests dynamically at iteration-level granularity.
- 🛡️ **Memory Manager:** Eliminates waste by reclaiming and recycling pages in real-time.



EXPERIMENTAL SETUP

Parameter	Configuration Details
Models	LLaMA-2-7B, Phi-3 Mini, OPT-13B
Hardware	NVIDIA A100 (80GB) / H100 (80GB)
Baseline	HuggingFace generate(), FasterTransformer
Metrics	Throughput (tokens/s), Latency, VRAM Utilization
Scenarios	Batch Sizes 1-128, Context Lengths up to 4k

RESULTS: THROUGHPUT SCALING



Measured in normalized tokens per second under heavy concurrent request load.

RESULTS: MEMORY EFFICIENCY

96%

VRAM UTILIZATION

Eliminating Waste

- ✓ **Zero Fragmentation:** vLLM reduces memory waste from 60-80% down to under 4%.
- 👤+ **Higher Concurrency:** Support for 2-4x more concurrent users on identical hardware.
- ⚡ **Scalability:** Linear scaling of memory usage with actual content generated.

KEY FINDINGS AND INSIGHTS



Systems Priority

Systems optimization (memory management) is just as critical as architectural innovation for LLM scaling.



Flexibility

PagedAttention enables dynamic sequence expansion and shared memory for parallel sampling with zero penalty.



Production Impact

vLLM's implementation significantly reduces the TCO (Total Cost of Ownership) for high-traffic AI services.

| LIMITATIONS & CHALLENGES

-  **Inference Only:** This is a serving optimization; it does not improve model accuracy or reduce training time.
-  **Complexity:** Higher engineering overhead to implement specialized PagedAttention kernels.
-  **Hardware Specifics:** Benchmarking gains are sensitive to GPU memory bandwidth and interconnects.
-  **Infrastructure:** Requires specialized deployment knowledge beyond simple library imports.

EXTENSIONS & FUTURE WORK

Edge Serving

Optimizing PagedAttention for consumer-grade NPUs.

Hybrid Engines

Synergy between vLLM and TensorRT-LLM frameworks.

Quantization

Integrating vLLM with 4-bit AWQ and GPTQ kernels.

VLM Support

Expanding paging to Vision-Language Model serving.

Conclusion

PagedAttention enables significantly more efficient LLM serving by eliminating memory fragmentation and maximizing GPU utilization. vLLM demonstrates that system-level memory management is the key to making Large Language Models practical and scalable for the real world.

Questions?

Efficient LLM Serving using vLLM and
PagedAttention

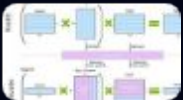
Nicholas Lisle

University of Central Florida –
CAP6614

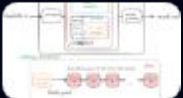
IMAGE SOURCES



https://pytorch.org/wp-content/uploads/2024/11/memory_leak_oom.jpg
Source: pytorch.org



https://miro.medium.com/0*oYTK9K1diMxApp6t
Source: medium.com



https://www.aleksagordic.com/blog/vllm/engine_constructor.png
Source: www.aleksagordic.com



https://media.istockphoto.com/id/2230807736/photo/futuristic-ai-data-center-interior.jpg?s=612x612&w=0&k=20&c=E2s9c_8IA4fBpNmX4gzn2qLnne4mXW1jhYOf9Essem8=
Source: www.istockphoto.com